2016 IEEE 17th International Conference on Information Reuse and Integration

Understanding Anti-forensic Techniques with Timestamp Manipulation

Dae-il Jang and Gail-Joon Ahn
Arizona State University
Tempe, USA
{*daeil.jang, gahn*}*@asu.edu*

Hyunuk Hwang and Kibom Kim
ETRI
Daejeon, South Korea
{*hhu, kabob*}*@nsr.re.kr*

***Abstract*—Numerous security incidents caused by malwares and hackers have recently utilized anti-forensic techniques to bypass analysis and detection. It is critical to build a knowledge base that would help understand such anti-forensic techniques. In this paper, we present a forensic analysis method to detect an anti-forensic technique which leverages timestamp manipulation in NTFS file system. Our approach analyzes how timestamp manipulation occurs in NTFS file system and also extracts some features to detect timestamp manipulation behaviors. We also evaluate our approach with several use cases and describe how our approach helps detect timestamp manipulation behaviors.**

***Keywords*-Anti-forensics; Timestamp Manipulation; NTFS**

I. Introduction

Digital forensics is a branch of forensic science encompassing the recovery and investigation of materials found in digital devices, often in relation to computer crime [1, 2]. The goal of digital forensics is to examine digital media for identifying, preserving, recovering, analyzing and presenting facts and opinions about the digital evidence [5]. Sophisticated digital forensic tools have helped identify, detect and resolve most of computer crime cases. However, recent cyber crimes have leveraged anti-forenisc techniques to bypass such forenisc analysis and detection tools. Therefore, it is inevitable to understand diverse anti-forensic techniques.

In this paper, we focus on one of anti-forensic techniques that relies on timestamp manipulation. Timestamp manipulation technique is to modify timestamp values for the purpose of making deceptively false representation of files. Timestamps are metadata that reflects when a file was last modified, last accessed, created or last written. These are known as MAC or MACE values in NTFS file system. Changes on such metadata can cause unexpected confusion and missing timelines in analyzing forensic envidences.

In order to understand diverse behaviors related to timestamp manipulation, we first analyze the features of anti-forensic techniques by examining raw images that are generated through several anti-forensic techniques. With the analysis results, we then propose and evaluate a forensic method for detecting timestamp manipulation in NTFS file system.

The rest of this paper is organized as follows. In Section 2, we discuss NTFS file system focused on $MFT and $Logfile structures. In Section 3, we elaborate timestamp changes caused by specific behaviors that would lead to timestamp detection rules addressed in Section 4. In Section 5, we briefly discuss the proof-of-concept implementation and our evaluation results and the paper is concluded in Section 6.

II. Structure of $MFT and $Logfile

In this section, we briefly overview important data structures in NTFS, which are used in our approach.

A. Structure of $MFT

The master file table (MFT) stores the information for each file required by NTFS [3]. A file has one or more MFT entry and each entry can contain one or more attributes in 1024 bytes [4]. Figure 1 illustrates the structure of MFT entry that contains several attributes.

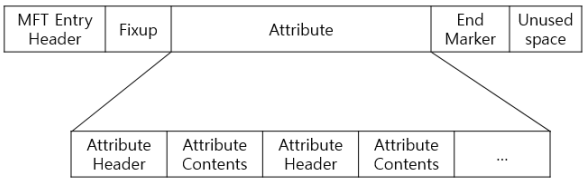

Figure 1. Structure of $MFT

First 48 bytes contain an MFT entry header. Structure of MFT entry header is shown in Table I. All $MFT entries have the signature value, `FILE`. Also, MFT entry includes an offset to fixup array that helps detect some abnormal changes, attributes relevant to file types[1], and $Logfile Sequence Number (LSN) to correlate with $Logfile.

NTFS file system views each file (or folder) as a set of file attributes. In other words, elements such as the file name, its security information, and even its data, are treated as file attributes. Table II enumerates all of file attributes currently defined in NTFS file system [6].

In this paper, we mainly use two attribute types, *$Standard_Information* and *$File_Name*. These two attribute types include four types of timestamps and we utilize eight types of time information to detect timestamp manipulation behaviors.

[1]It includes normal file, symbolic file, bitmap file, and so on.

978-1-5090-3207-5/16 $31.00 © 2016 IEEE
DOI 10.1109/IRI.2016.94

Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:20:15 UTC from IEEE Xplore. Restrictions apply.

Table I
$MFT ENTRY HEADER

Byte Range	Description	Essential
0 - 3	Signature ("FILE")	No
4 - 5	Offset to fixup array	Yes
6 - 7	Number of entries in fixup array	Yes
8 - 15	$LogFile Sequence Number (LSN)	No
16 - 17	Sequence Number	No
18 - 19	Link count	No
20 - 21	Offset to first attribute	Yes
22 - 23	Flags (in-use and directory)	Yes
24 - 27	Used size of MFT entry	Yes
28 - 31	Allocated size of MFT Entry	Yes
32 - 39	File reference to base record	No
40 - 41	Next attribute id	No
42 - 43	Align to 4B boundary	No
44 - 47	Number of this MFT Entry	No

Table II
ATTRIBUTE LIST

Attribute ID	Attribute Type
16 (0x10)	Standard Information
32 (0x20)	Attribute List
48 (0x30) / 64 (0x40)	File Name
64 (0x40)	Volume version Object ID
80 (0x50)	Security Descriptor
96 (0x60)	Volume Name
112 (0x70)	Volume Information
128 (0x80)	Data
144 (0x90)	Index Root
160 (0xA0)	Index Allocation
176 (0xB0)	Bitmap
192 (0xC0)	Symbolic link
192 (0xC0)	Reparse Point
208 (0xD0)	EA information
224 (0xE0)	EA
256 (0xF0)	Logged Tool Stream

B. Structure of $Logfile

$LogFile is an NTFS journal file to record all changes of files called system transaction records that include creation, deletion and modification of data or MFT entry [7, 8]. It is used to recover files from system error or unexpected conditions. The $Logfile records all transactions to a record unit that has the unique $Logfile Sequence Number (LSN) [2]. The record also contains `Redo` (working data) and `Undo` (before working data) data. The structure of $LogFile consists of two areas: restart and logging. Restart area refers to the last/current operation record and is located in the first 2 pages at the address 0x0000 $\sim$ 0x2000. The restart page header signature is `RSTR` with 13 attributes. Logging area contains actual operation records. This area is divided into *buffer page area* and *normal page area*. The first two pages (0x2000 $\sim$ 0x4000) in logging area is buffer page area. Operation records are written in the buffer page area. When the page is fully stored, the contents are moved to normal page area. Normal page area stores records sequentially. If this area is full, the records are overwritten from the start address 0x4000. The page to store operation records consists of one header and multiple operation records. Page header has the metadata such as magic signature value, `RCRD` and 11 attributes including the last LSN, next record offset, and the last end LSN.

[2]LSN is increased sequentially.

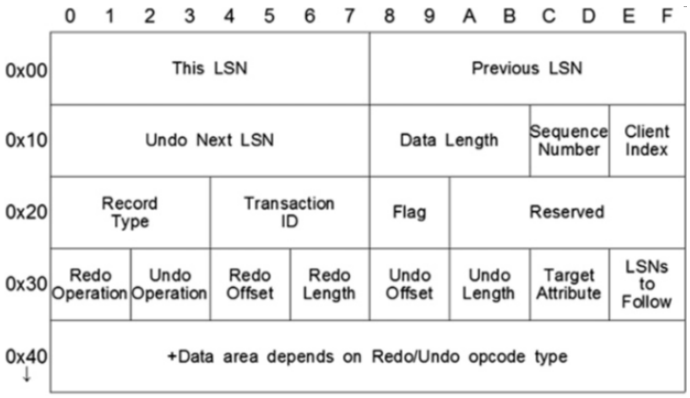

Figure 2. Structure of $Logfile

Figure 2 illustrates the structure of $Logfile. `This LSN` and `Previous LSN` attributes indicate LSN of the current record and the previous record to the related pages, respectively. `Undo LSN` is LSN information relevant to undo operations. `Data Length` is the size of record from redo operation field to the end of the record. Table III lists operation codes for redo and undo operations [9].

When we analyze $Logfile to extract timestamp of files, we focus on redo and undo operations to understand the innerworkings in file management. For example, the record order of file creation is as follows.

1) **0x15/0x16**:
   Set Bits In Nonresident Bit Map/Clear Bits In Nonresident Bit Map
2) **0x00/0x03**:
   Noop/Deallocate File Record Segment
3) **0x0E/0x0F**:
   Add Index Entry Allocation/Delete Index Entry Allocation
4) **0x02/0x00**:
   Initialize File Record Segment/Noop
5) **0x1B/0x01**:
   Forget Transaction/Compensation Log Record

We first get an MFT entry number from redo data of 0x15/0x16. Four bytes of redo data include the MFT entry number of the target MFT record. We then get timestamps and file name from redo data of 0x02/0x00. This redo data is a content of MFT record so it also allows us to obtain all MFT entries that include timestamps and file names.

Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:20:15 UTC from IEEE Xplore. Restrictions apply.

Table III
$LOGFILE OPERATION CODE

Operation Type	Operation Value
Noop	0x00
CompensationlogRecord	0x01
InitializeFileRecordSegment	0x02
DeallocateFileRecordSegment	0x03
WriteEndofFileRecordSegement	0x04
CreateAttribute	0x05
DeleteAttribute	0x06
UpdateResidentValue	0x07
UpdataeNonResidentValue	0x08
UpdateMappingPairs	0x09
DeleteDirtyClusters	0x0A
SetNewAttributeSizes	0x0B
AddindexEntryRoot	0x0C
DeleteindexEntryRoot	0x0D
AddIndexEntryAllocation	0x0F
SetIndexEntryVenAllocation	0x12
UpdateFileNameRoot	0x13
UpdateFileNameAllocation	0x14
SetBitsInNonresidentBitMap	0x15
ClearBitsInNonresidentBitMap	0x16
PrepareTransaction	0x19
CommitTransaction	0x1A
ForgetTransaction	0x1B
OpenNonresidentAttribute	0x1C
DirtyPageTableDump	0x1F
TransactionTableDump	0x20
UpdateRecordDataRoot	0x21

III. TIMESTAMP CHANGES BY NORMAL BEHAVIORS

Time stamp manipulation in malware has two cases. One case is to change the timestamp for operating system. This is relatively easy to detect because it changes the information about all behaviors related to time such as file generation, access, modification, log recording time, and all other behaviors. Other case is to manipulate MACE values of files. Most malware packages change the $Standard_Information attribute by using some API calls. For that case we can detect such a behavior by comparing the $Standard_Information and the $File_Name attributes. However, the malware may also change those two attributes so we can only find a signature of the executed malware. In addition to MACE changes, malware can change the timestamp value based on specific malware configuration setting or certain system files [3].

A. Timestamp changing rule

We analyzed MFT for each behavior that can change the timestamp in a file system. Based on our analysis, we categorize 14 patterns of file operations as follows: Creation, Copy, Move Data in MFT, Move Data in cluster, Move from other partition using Drag and Drop, Move from other partition using Cut and Paste, File property change, File modification through notepad, File modification through MSOffice, File name change, Internet Download, Uncompressed, Copy from other file system, and Install program. We use the following definitions to define these behaviors. These definitions are borrowed from [9].

- Definition I. Operation time: T_{op}
  - The time when a user commands operations on a file for some behaviors
- Definition II. Original time: T_{src}
  - Original timestamp before an operation is performed except for the file creation
- Definition III. System time: T_{sys}
- Definition IV. Execution time: Δ
  - The time it takes to execute the command
- Definition V. MACE
  - m : Modification time
  - a : Access time
  - c : Creation time
  - e : MFT Entry modification time
- Definition VI. The current state of timestamp value for $STANDARD_INFORMATION and $FILE_NAME: $(t_x^{c,m,e,a})_{SI}$ and $(t_x^{c,m,e,a})_{FN}$, where $x \in \{op, src, sys\}$.
  - It denotes the current states of timestamp values in the $STANDARD_INFORMATION (SI) attribute and $FILE_NAME (FN) attribute in MFT.

Based on the above-mentioned definitions, we now define 14 patterns as follows:

Creation: It changes all of the timestamps both in $SI and $FN. In particular, we found two cases in $SI. When a user uses the mouse's right click to create a new file or rename the file name, the timestamps do not have the same value. We noticed that it takes some time for modifying MFT entry in $SI attribute. Another case is that the same time values are listed in $SI when a user does not edit the file name. We define these behaviors as follows:

- $t_{op} = (t_{op}^{c,m,e,a})_{SI} = (t_{op}^{c,m,e,a})_{FN}$, if the right click was used or file was renamed.
- $t_{op} = (t_{op}^{c,m,a} < t_{op}^{e} + \Delta)_{SI} = (t_{op}^{c,m,e,a})_{FN}$

Copy: When a user copies some files, creation time and access time of $SI are set to the time T_{op}. MFT entry modification time is set to $t_{op}^{e} + \Delta$ and all timestamps in $FN are set to T_{op}. We define these behaviors as follows:

- $(t_{src}^{m} < t_{op} = t_{op}^{c,a} \leq t_{op}^{e} + \Delta)_{SI}$, where if $\Delta \cong 0$ then $t_{op}^{e} = t_{op}$, $t_{op} = (t_{op}^{c,m,e,a})_{FN}$

Copy from difference file system: This operation yields the same result as copy operation. However, if the source file system does not support the micro time, we noticed that the micro time is set to 0.

Move Data in MFT: If we move a file to some other directories in the same volume, MFT modification time is changed to $t_{op}^{e} + \Delta$ in $SI but $FN is not changed. The behavior is defined as follows:

[3]Some malwares have leveraged the timestamp of `calc.exe` to change the timestamp of files that malwares created or modified.

Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:20:15 UTC from IEEE Xplore. Restrictions apply.

- $(t_{src}^{c,m,a} \leq t_{op}^{e} + \Delta)_{SI}$, $(t_{src}^{c,m,a,e})_{FN}$, where if $\Delta \cong 0$ then $t_{op}^{e} = t_{op}$

Move Data in Cluster: This operation is similar to the above case but access time is changed instead of MFT modification time. The behavior is defined as follows:

- $(t_{src}^{c,m,a} < t_{op} = t_{op}^{e})_{SI}$, $(t_{src}^{c,m,a,e})_{FN}$

Move from other partition using Drag and Drop: This operation changes creation time, access time and MFT modification time in $SI and $FN fields are all set to the operation time. These behaviors are defined as follows:

- $(t_{src}^{m} < t_{op} = t_{op}^{c,a} \leq t_{op}^{e} + \Delta)_{SI}$, $t_{op} = (t_{op}^{c,m,a,e})_{FN}$

Move from other partition using Cut and Paste: This operation changes modification time and MFT modification time in $SI and $FN fields are all set to the operation time. These behaviors are defined as follows:

- $(t_{src}^{c,m} < t_{op} = t_{op}^{a} \leq t_{op}^{e} + \Delta)_{SI}$, $t_{op} = (t_{op}^{c,m,a,e})_{FN}$

File property / name change: This operation mainly changes MFT modification time. The behavior is defined as follows:

- $(t_{src}^{c,m,a} \leq t_{op}^{e} + \Delta)_{SI}$, $(t_{src}^{c,m,a,e})_{FN}$

File modification using Notepad: When a user modifies a file using notepad, modification time and MFT modification time in $SI are changed and all $FN timestamps remain intact. The behavior is defined as follows:

- $(t_{src}^{c,a} < t_{op} = t_{op}^{m} \leq t_{op}^{e} + \Delta)_{SI}$, $(t_{src}^{c,m,a,e})_{FN}$

File modification using MSOffice: When a user modifies a file using MSOffice, all timestamps in $SI and $FN are set to T_{op} except for creation time. These behaviors are defined as follows:

- $(t_{src}^{c} < t_{op} = t_{op}^{m,a} \leq t_{op}^{e} + \Delta)_{SI}$, $(t_{src}^{c} < t_{op} = t_{op}^{m,a})_{FN}$

File download with internet: When a user downloads some files via the Internet, all timestamps are changed. In this case, modification time and MFT modification time in $SI and $FN are changed. These behaviors are defined as follows:

- $t_{op1} = (t_{op1}^{c,m,e,a})_{SI} = (t_{op1}^{c,m,e,a})_{FN}$
- $t_{op2} = (t_{op1}^{c,a} < t_{op2}^{m,e} + \Delta)_{SI}$, $(t_{op1}^{c,a} < t_{op2}^{m,e})_{FN}$

Uncompressed with 7zip: When a user uses 7zip for uncompressing the file, it generates the following two cases:

- If a file has all types of timestamps, $(t_{src}^{c,m,a} \leq t_{op}^{e} + \Delta)_{SI}$, $(t_{op}^{c,m,a,e})_{FN}$
- Else one type of timestamp, $(t_{src}^{m} < t_{op}^{c,a} \leq t_{op}^{e} + \Delta)_{SI}$, $(t_{op}^{c,m,a,e})_{FN}$

Uncompressed with Alzip: Alzip uses only modification time if the compressed file has other time information. Hence, this behavior is quivalent to the second case of 7zip.

- $(t_{src}^{m} < t_{op}^{c,a} \leq t_{op}^{e} + \Delta)_{SI}$, $(t_{op}^{c,m,a,e})_{FN}$

Install a program: This operation is similar to the uncompression operation. It changes access time and MFT modification time in $SI and all $FN timestamps are changed. Otherwise, modification time is not changed.

- If it includes all types of timestamps, $(t_{src}^{m,c} < t_{op}^{a} \leq t_{op}^{e} + \Delta)_{SI}$, $(t_{op}^{c,m,a,e})_{FN}$
- otherwise, $(t_{src}^{m} < t_{op}^{c,a} \leq t_{op}^{e} + \Delta)_{SI}$, $(t_{op}^{c,m,a,e})_{FN}$

IV. Anti-Forensic Detection Method

Based on the definitions in Section 3, we introduce several rules to detect timestamp modification. Our method is based on the patterns that we described in the previous sections. The following rules are defined to leverage those patterns.

Rule 1: Future time All timestamps in $SI and $FN should be less than or equal to t_{op}^{e} and the timestamp does not exceed the system time.

- Rule 1-1: $t_{op}^{e} < t_{op}^{c,m,a}$
- Rule 1-2: $(t_{src}^{m,a,c,e})_{(SI|FN)} > t_{sys}$

Rule 2: $SI information modification If an attacker changes a timestamp using timestamp manipulation tools, those tools do not change $FN timestamp except for a tool called SetMACE. Hence, $FN timestamp can be a baseline to detect $SI information modification.

- Rule 2-1: $(t_{op}^{e})_{SI} < (t_{op}^{e})_{FN}$
- Rule 2-2: $(t_{op}^{c,m,a})_{SI} > (t_{op}^{c,m,a})_{FN}$

Rule 3: $FN information modification If an attacker changes $FN timestamp using SetMACE, the following detection rule is used.

- Rule 3-1: $(t_{op}^{c,a})_{FN} > (t_{op}^{m,e})_{FN}$
- Rule 3-2: $(t_{op}^{m})_{FN} \neq (t_{op}^{e})_{FN}$

A. Correlation with $MFT and $LogFile

$MFT contains the information about all files and directories–even if the file or directory is deleted. We can also get MACE values of files through $MFT. However, attackers can manipulate this information. We cannot detect timestamp manipulation by simply using $MFT analysis. Therefore, we also correlate attributes between $MFT and $LogFile. $LogFile writes file system events and these logs have some events that cannot be found in $MFT. For example, a history of the deleted files or a specific file's behavior can be obtained. $LogFile is also in MFT and uses the NTFS journal. It has standard file attributes and stores the log data to record metadata changes to the volume. It means each log data records file/directory creation, deletion, change, and MFT entry change. However, this record may not contain every event of each file so it is critical to extract all relevant events related to the record.

Rule 4: comparing creation time with $MFT and $Logfile, For the case that an attacker changes the creation time on $FN and $SI using manipulation tools, we need the following rule:

- Rule 4: $(t^{c})_{\$Logfile} \neq (t_{op}^{c})_{\$FN/\$SI}$

Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:20:15 UTC from IEEE Xplore. Restrictions apply.

Table IV
TIMESTAMP MANIPULATION: TOOLS AND CHANGED FIELD

File name	Used tool	Changed field	Time value
XTST_CT.txt	XTST	Create time in $SI	Random
XTST_MT.txt	XTST	Modification time in $SI	Random
XTST_AT.txt	XTST	Access time in $SI	Random
XTST_ALL.txt	XTST	MACE time in $SI	Random
TS_CT.txt	TimeStomp	Create time in $SI	Random
TS_MT.txt	TimeStomp	Modification time in $SI	Random
TS_AT.txt	TimeStomp	Access time in $SI	Random
TS_ALL.txt	TimeStomp	MACE time in $SI	Random
SM_CT_SI.txt	SetMACE	Create time in $SI	Random
SM_CT_FN.txt	SetMACE	Create time in $FN	Random
SM_CT_ALL.txt	SetMACE	Create time in $SI and $FN	Random, same value
SM_MT_SI.txt	SetMACE	Modification time in $SI	Random
SM_MT_FN.txt	SetMACE	Modification time in $FN	Random
SM_MT_ALL.txt	SetMACE	Modification time in $SI and $FN	Random, same value
SM_AT_SI.txt	SetMACE	Access time in $SI	Random
SM_AT_FN.txt	SetMACE	Access time in $FN	Random
SM_AT_ALL.txt	SetMACE	Access time in $SI and $FN	Random, same value
SM_ALL_SI.txt	SetMACE	MACE time in $SI	Random
SM_ALL_FN.txt	SetMACE	MACE time in $FN	Random
SM_ALL_ALL.txt	SetMACE	MACE time in $SI and $FN	Random, same value
SM_ALL_SI_other.txt	SetMACE	MACE time in $SI	Other file
SM_ALL_FN_other.txt	SetMACE	MACE time in $FN	Other file
SM_ALL_ALL_other.txt	SetMACE	MACE time in $SI and $FN	Other file

B. Using deleted file information

Malware typically uses data cluster area which is sometimes used by deleted files. Therefore, if we get the information of deleted files from $MFT, it helps (i) determine a start address of data cluster for deleted files which a malware program may overwrite and (ii) detect pontential timestamp modification.

Rule 5: comparing file location with deleted file and new If an attacker manipulates a timestamp and some MFT entries, we can use the following rule.

- Rule 5: $((t^e)_{SI}, (t_{op})_{FN})_{File} < ((t^e)_{SI})_{Deleted}$

C. Using features of NTFS and manipulation tools

NTFS and timestamp manipulation tools have many features. One feature deals with the micro time. NTFS file system uses the micro time consisting of milliseconds and nanoseconds. The most normal timestamps can be shown in micro time. However, in the NTFS, we observed cases where the micro time has no value. One case where this would occur is with moving files and copying files from FAT systems. FAT systems do not support the micro time, so if a user moves or copies a file from FAT, the access time is set to zero in micro seconds. For other case, if some files are generated on FAT and a user downloads, decompresses, and installs such files, some parts of $SI timestamps are set to zero in micro seconds. Also, most timestamp manipulation tools set to zero in microseconds. Hence, we can also use the micro time to detect time modification.

Rule 6: Microtime Check For the case that an attacker manipulates a timestamp using XTST or Timestomp for MFT Entry modification time, we use the following rule.

- Rule 6:$((t^e)_{SI})_{Microtime} = Zero$

Rule 7: Comparing creation time If an attacker manipulates a timestamp to copy other file, the creation time is compared.

V. IMPLEMENTATION AND EXPERIMENT RESULT

To demonstrate the feasibily of our approach, we built a proof-of-concept prototype to utilize the rules and tasks described in Section 4. Our tool consists of three modules; physical drive/image parser, NTFS parser for file instances, and anomaly detector for timestamp manipulation. Physical drive/image parser analyzes and finds logical volumes and classifies a file system. If a volume is NTFS, our tool parses file instances in NTFS file system. In order to perform this task, the NTFS parser consists of two modules; $MFT parser and $Logfile parser. $MFT parser extracts timestamps of $SI and $FN for all files including the deleted files. $Logfile parser extracts information of all file creation events. The anomaly detector detects abnormal timestamps based on our rules, correlation analysis and comparison between Volume Shadow Copy (VSC) and $Logfile.

To evaluate our tool, we also generated several raw images and modified NTFS storage by using timestamp manipulation tools including SetMACE, TimeStomp, and XTST. Table IV shows the information of timestamp manipulated files. Our experiment results are shown in Table V. Our results indicate that the most efficient method is to check the timestamp of file's behavior based on VSC and $Logfile while our tool could detect all cases by applying rules ans tasks defined in Section 4.

Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:20:15 UTC from IEEE Xplore. Restrictions apply.

Table V
EXPERIMENT RESULTS ON TIMESTAMP MANIPULATION DETECTION

File name	RuleBase Detector	FileBehavior TimeChecker	DeletedFile InfoChecker	Timestamp Equality Checker
XTST_CT.txt	X	O	X	X
XTST_MT.txt	X	O	X	X
XTST_AT.txt	X	O	X	X
XTST_ALL.txt	O	O	X	X
TS_CT.txt	X	O	X	X
TS_MT.txt	X	O	X	X
TS_AT.txt	X	O	X	X
TS_ALL.txt	O	O	X	X
SM_CT_SI.txt	X	O	X	X
SM_CT_FN.txt	O	O	X	X
SM_CT_ALL.txt	O	O	X	X
SM_MT_SI.txt	X	O	X	X
SM_MT_FN.txt	O	O	X	X
SM_MT_ALL.txt	O	O	X	X
SM_AT_SI.txt	X	O	X	X
SM_AT_FN.txt	O	O	X	X
SM_AT_ALL.txt	O	O	X	X
SM_ALL_SI.txt	O	O	X	X
SM_ALL_FN.txt	O	O	X	X
SM_ALL_ALL.txt	X	O	X	X
SM_ALL_SI_other.txt	O	O	X	O
SM_ALL_FN_other.txt	O	O	X	O
SM_ALL_ALL_other.txt	X	O	X	O

VI. CONCLUSION

Cybercrimes can occur with various digital devices such as computers, cellphones, smart TVs and so on. Digital forensic research has investigated diverse analysis techniques to generate useful and actionable intelligence from digital evidences related to such cybercrimes. However, the advent of anti-forensic techniques significantly neutralized digital forensic examination. For example, various timestamp modification tools are capable of modifying time attributes of system files, user files/directories and other binaries so the forensic examiners may easily omit essential clues for resolving cybercrime cases without rigorous analysis of all files.

In this paper, we have introduced a forensic analysis method to detect an anti-forensic technique which leverages timestamp manipulation in NTFS file system. We analyzed the features of anti-forensic techniques by examining raw images that are generated through several anti-forensic techniques and then proposed rules and additional tasks for detecting timestamp manipulation in NTFS file system. Our evaluation results showed how effectively our approach could detect time manipulation behaviors.

For the future work, we believe the timeline generation with our tool is critical to efficiently identify any missing or suspicious events. In addition, we are planning to expand our approach to support other file systems with an interactive tool that can help specify and test new rules for diverse file systems.

REFERENCES

[1] Brian Carrier. Defining digital forensic examination and analysis tools, *International Journal of Digital Evidence*, 2002.

[2] Mark Reith Clint , Mark Reith , Clint Carr , and Gregg Gunsch. An examination of digital forensic models, *International Journal of Digital Evidence*, 2002.

[3] L. Naiqi, W. Yujie, and H. QinKe. Computer Forensics Research and Implementation Based on NTFS File System, *Proc. of ISECS International Colloquium on Computing, Communication, Control, and Management (CCCM'08)*, 2008.

[4] Alazab, M.; Venkatraman, S., and Watters, P. Effective digital forensic analysis of the NTFS disk image, *Ubiquitous Computing and Communication Journal*, 2009.

[5] Michael G. Noblett, Mark M. Pollitt, and Lawrence A. Presley, Recovering and examining computer forensic evidence, *Forensic Science Communications*, vol 2, 2000.

[6] NTFS documentation, http://www.linux-ntfs.org.

[7] C. Swenson, R. Phillips, and S. Shenoi. File system journal forensics. *Advances in digital forensics III*, IFIP international federation for information processing, vol. 242. Boston: Springer; 2007.

[8] Gyu-Sang Cho. An analysis of NTFS journal file for a computer forensic, *Digital Forensic Research*, 2009.

[9] Gyu-Sang Cho. A computer forensic method for detecting timestamp forgery in NTFS, *Computers and Security*, 2013.

Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:20:15 UTC from IEEE Xplore. Restrictions apply.